



Tecnológico de Monterrey

**Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla**

Materia: Implementación de robótica inteligente

Clave: TE3002B.501

Grupo: 501

Profesor:

Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios
Alfredo García Suárez
Cesar Torres Huitzil

Alumnos

Omar Perez Dominguez | A01738306
Ezequiel Luna Trejo | A01738153
Antonio Méndez Rodríguez | A01738269

24 de Mayo De 2026

Resumen:

El siguiente reporte documenta el diseño, entrenamiento e integración en el ecosistema ROS 2 de un sistema de percepción visual basado en la arquitectura YOLOv8 Nano, orientado a la identificación en tiempo real de siete clases de señales de tránsito para la plataforma robótica móvil Puzzlebot. El objetivo central consistió en desarrollar un modelo computacional robusto capaz de procesar inferencias con alta precisión y baja latencia.

Para superar las limitaciones de un *dataset* base de 3,500 imágenes y evitar el sobreajuste del modelo, el pipeline metodológico implementó técnicas de regularización como el paro anticipado (*early stopping*) y un proceso estocástico de aumentación de datos dinámica en RAM. Esta estrategia permitió simular computacionalmente las perturbaciones fotométricas y vibraciones mecánicas propias de la pista física de pruebas sin saturar el almacenamiento local.

Los resultados experimentales demuestran una convergencia óptima de los pesos neuronales, garantizando una alta tasa de cuadros por segundo (FPS) y una latencia mínima en la detección. Esto valida la eficiencia del sistema de percepción.



Imagen 1. Ejemplo de la red neuronal funcionando

Objetivos

Objetivo general

Diseñar, entrenar e integrar en el ecosistema ROS 2 un modelo de visión computacional robusto basado en la arquitectura YOLOv8 Nano, implementando una arquitectura de procesamiento distribuido para identificar y clasificar en tiempo real siete clases de señales de tránsito, garantizando una navegación autónoma eficiente y de baja latencia en la plataforma robótica móvil Puzzlebot.

Objetivos Específicos

- Creación del dataset: Estructurar y exportar un dataset unificado de 3,500 imágenes debidamente etiquetadas en formato de cajas delimitadoras estándar de YOLOv8 a través de la plataforma Roboflow.
- Configuración e infraestructura de cómputo: Desplegar e implementar entornos paralelos de desarrollo en la nube (Google Colab) y de forma nativa en hardware local, asegurando la compatibilidad de las librerías PyTorch y Ultralytics.
- Optimización de robustez mecánica y lumínica: Configurar un pipeline de data augmentation dinámico en RAM operando en rangos para emular perturbaciones del robot real.
- Mitigación del sobreajuste: Implementar hiperparámetros estrictos de regularización y un gradiente de aprendizaje inicial controlado para suavizar la convergencia de pesos.
- Despliegue de Arquitectura Distribuida en ROS 2: Desarrollar e implementar un nodo suscriptor en ROS 2 para la estación de trabajo local que intercepte el flujo de video transmitido por el nodo publicador del Puzzlebot a través de la red local inalámbrica.

Introducción:

En el marco del desarrollo de sistemas de detección visual para la navegación autónoma del Puzzlebot, este reporte detalla la implementación de un sistema de detección de objetos en tiempo real. El reto principal consistió en dotar al robot de la capacidad de identificar y clasificar siete señales de tránsito diferentes bajo condiciones físicas variables (iluminación, distancia y vibraciones), integrando algoritmos de estado del arte en el ecosistema distribuido de ROS 2.

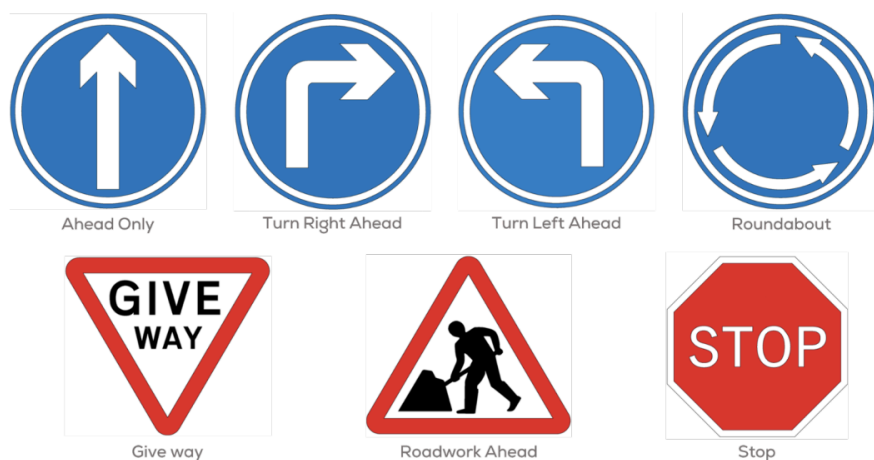


Imagen 2. Señales de tránsito a detectar

1. Deep Learning (Aprendizaje Profundo)

¿Qué es?: Es una subdisciplina de la Inteligencia Artificial que utiliza redes neuronales artificiales con múltiples capas ocultas para extraer jerarquías de características complejas a partir de datos crudos (como matrices de píxeles), eliminando la necesidad de programar reglas de extracción manuales.[1]

¿Cómo se utilizó?: Sirvió como el paradigma fundamental para el desarrollo del sistema de visión. En lugar de utilizar técnicas clásicas de visión computacional (como umbralización de colores que fallarían con los cambios de luz en la pista), se optó por un enfoque basado en datos, permitiendo que el sistema "aprendiera" por sí mismo los patrones visuales que definen a cada una de las siete señales de tránsito.

2. Red Neuronal Convolutacional (CNN)

¿Qué es?: Es un tipo específico de red neuronal artificial diseñada explícitamente para procesar datos con topología de cuadrícula, como las imágenes. Utilizan operaciones matemáticas de convolución para identificar bordes, texturas y formas geométricas, volviéndose invariantes a la posición del objeto en la imagen.[2]

¿Cómo se utilizó?: Constituye la arquitectura base del modelo entrenado. Las capas convolucionales se encargaron de procesar los fotogramas capturados por la cámara del Puzzlebot, extrayendo los mapas de características necesarios para distinguir entre señales visualmente similares.

3. YOLO (You Only Look Once)

¿Qué es?: Es un algoritmo de detección de objetos de última generación que reformula el problema de visión como un único problema de regresión. A diferencia de modelos más antiguos, YOLO procesa la imagen completa en una sola pasada a través de la red neuronal, prediciendo simultáneamente las cajas delimitadoras (bounding boxes) y las probabilidades de clase.[3]

¿Cómo se utilizó?: Se seleccionó e implementó la arquitectura YOLOv8 Nano (yolov8n.pt) debido a su ligereza computacional. Se integró en un nodo de ROS 2 utilizando la librería cv_bridge para interceptar el flujo de video del robot y realizar la inferencia en tiempo real, garantizando una detección precisa con la latencia mínima que requiere la navegación autónoma.

4. Épocas de entrenamiento y paro anticipado (Early Stopping)

¿Qué es?: Una época (epoch) representa un ciclo completo en el que la red neuronal procesa todo el conjunto de datos de entrenamiento una vez. El paro anticipado es una técnica de regularización que detiene el entrenamiento automáticamente si el modelo deja de mejorar, evitando el sobreajuste (overfitting).[2]

¿Cómo se utilizó?: El modelo se configuró para entrenar durante un máximo de 100 épocas (epochs=100) para asegurar una convergencia profunda de los pesos. Sin embargo, se implementó un hiperparámetro de paciencia de 20 ciclos (patience=20), lo que permitió que el entrenamiento se detuviera de forma inteligente en el momento óptimo, garantizando que el modelo aprendiera a generalizar las señales en lugar de solo memorizar las 3,500 fotos del dataset original.

5. Data Augmentation

¿Qué es?: Es una técnica que expande artificialmente el tamaño y la diversidad de un conjunto de datos de entrenamiento aplicando alteraciones matemáticas a las imágenes originales antes de pasarlas por la red neuronal.[4]

¿Cómo se utilizó?: Se implementó dinámicamente en la memoria RAM durante el proceso de entrenamiento. Esto fue vital para robustecer el modelo frente a las perturbaciones del mundo real sin necesidad de fotografiar físicamente miles de variaciones de las señales. Preparó al modelo para enfrentar las condiciones imperfectas de la pista de pruebas.

6. Transformaciones fotométricas y geométricas

¿Qué es?: Son las operaciones matemáticas específicas que alteran los píxeles durante la aumentación de datos. Las transformaciones geométricas alteran la posición o escala (rotación, traslación), mientras que las fotométricas alteran el espacio de color y la iluminación.[3]

¿Cómo se utilizó?: Se configuró el script de entrenamiento con hiperparámetros estrictos para emular la cinemática del Puzzlebot:

- Brillo (hsv_v=0.4): Se alteró la iluminación en un $\pm 40\%$ para simular sombras y luces cenitales en la pista.
- Rotación (degrees=15.0): Se aplicaron giros de hasta 15° para emular la inclinación del chasis al pasar por irregularidades.
- Escala y Traslación (scale=0.5, translate=0.1): Simularon las aproximaciones del robot hacia la señal y las vibraciones de la cámara.
- Mosaico (mosaic=1.0): Fusionó múltiples imágenes para obligar al modelo a detectar señales en fondos ruidosos.
- Restricción de Espejo (fliplr=0.0): Se apagó deliberadamente el espejado horizontal, ya que invertir una señal de tránsito (como una flecha de dirección) destruiría su significado semántico y confundiría al controlador del robot.

Solución del problema

El desarrollo, entrenamiento y validación del sistema de percepción visual se ejecutó a través de un pipeline secuencial. A continuación, se desglosan las fases operativas y los resultados de cada etapa, comenzando por la construcción del dataset.

1. Generación y Estructuración del Dataset Base

La primera fase crítica del proyecto consistió en la creación de un conjunto de datos propio y altamente representativo del entorno de operación. Se recolectaron un total de 3,500 imágenes fotográficas aproximadamente, distribuidas en 500 muestras para cada una de las siete clases de señales de tránsito.

para la captura total del dataset se utilizó la cámara instalada a bordo del Puzzlebot. Al usar la misma cámara, se garantizó que las imágenes de entrenamiento contuvieran los parámetros intrínsecos exactos del sensor como la distorsión geométrica del lente, la distancia focal y el perfil específico de ruido óptico, que el modelo enfrentaría posteriormente durante la navegación en vivo.

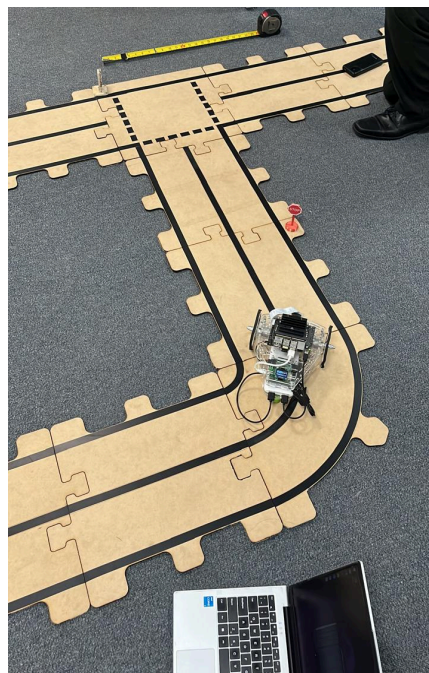


Imagen 3. Puzzlebot tomando fotos para el dataset

2. Anotación de datos y estructuración en Roboflow

Una vez concluida la recolección fotográfica, el pipeline avanzó hacia la fase de etiquetado. Para garantizar la máxima precisión geométrica en la definición de las Regiones de Interés (ROI), este proceso se llevó a cabo de manera estrictamente manual utilizando la plataforma Roboflow.

A cada una de las 3,500 imágenes se le asignó una caja delimitadora (bounding box) a la señal de tránsito correspondiente. Este rigor en el etiquetado manual fue imperativo; unas cajas delimitadoras demasiado holgadas habrían introducido exceso de ruido de fondo en la región que la red neuronal analiza como "objeto", mientras que cajas demasiado estrechas habrían recortado características morfológicas clave de las señales, afectando negativamente la precisión de la inferencia.

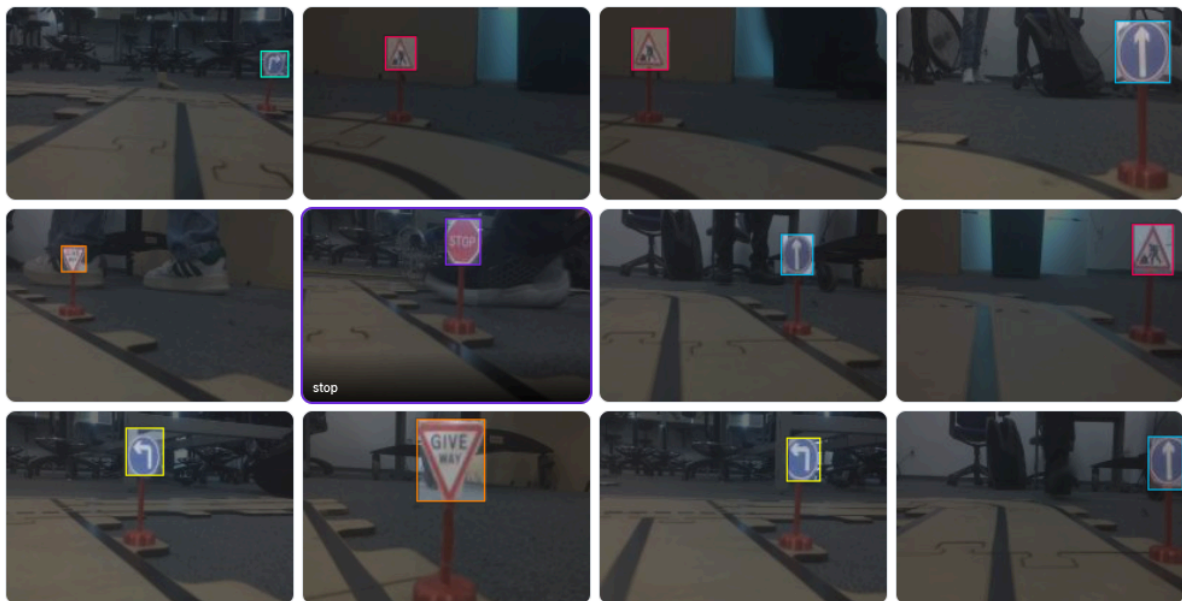


Imagen 4. Ejemplo de las señales de tránsito con el bounding box

Tras finalizar el etiquetado de las siete clases, se hizo la exportación del dataset. En esta etapa, la plataforma Roboflow ejecutó tres procesos fundamentales para la preparación del entorno de Deep Learning:

- Partición estocástica de datos (Data Splitting): Para poder evaluar la capacidad de generalización de la red y evitar el sobreajuste, el conjunto total de imágenes no se utilizó completamente para entrenar. Roboflow dividió de forma aleatoria y estratificada el dataset en dos subconjuntos aislados: el conjunto de Entrenamiento (utilizado para el cálculo del gradiente y el ajuste de pesos) y el conjunto de Validación (un grupo de imágenes que la red nunca ve durante la optimización, utilizado exclusivamente al final de cada época para evaluar la precisión real del modelo y activar el mecanismo de Early Stopping).
- Conversión al formato nativo de YOLOv8 : El ecosistema de Ultralytics requiere una topología de directorios muy específica (separando images/ y labels/). Roboflow transformó las coordenadas en píxeles de las cajas delimitadoras dibujadas manualmente a archivos de texto plano (.txt). En este formato, cada imagen cuenta con su respectivo archivo de texto donde la anotación se representa mediante un vector numérico: [id_clase, x_centro, y_centro, ancho, alto]. Es importante destacar que estos valores geométricos se exportaron normalizados en un rango de 0 a 1. Esta abstracción matemática permite que las capas convolucionales procesen las proporciones del objeto independientemente de la resolución original de la fotografía.
- Generación del data.yaml: El subproducto de mayor relevancia arquitectónica en esta fase fue la creación automática del archivo de configuración YAML. Este archivo funge como el índice maestro y el puente de intercomunicación entre los tensores de datos crudos y el modelo de PyTorch. En su estructura interna, el data.yaml define las rutas absolutas o relativas hacia las carpetas de entrenamiento y validación, establece el número total de tensores de salida necesarios (nc: 7), y mapea el arreglo de identificadores numéricos hacia sus etiquetas semánticas legibles por el humano (names: ['alto', 'siga', 'curva', ...]).

3. Configuración y ejecución del entrenamiento computacional

La fase culminante del pipeline consistió en la optimización de la red neuronal convolucional. Para llevar a cabo este proceso, se desarrolló un script en Python implementando la librería ultralytics, el cual orquestó la ingesta de datos, la configuración de hiperparámetros y la ejecución del entrenamiento acelerado por hardware (device=0).

- Aprendizaje transferido (Transfer Learning) y arquitectura base

En lugar de inicializar la red neuronal con pesos aleatorios, el proceso comenzó cargando los pesos preentrenados del modelo base yolov8n.pt. Esta técnica de Aprendizaje Transferido permitió que el modelo aprovechara los mapas de características previamente aprendidos como la detección básica de bordes, geometrías y contrastes, enfocando el esfuerzo computacional exclusivamente en realizar el ajuste fino para el dominio específico de las siete señales de tránsito del Puzzlebot. La elección de la variante "Nano" fue deliberada para garantizar una baja latencia durante la inferencia en tiempo real.

- Hiperparámetros de optimización y prevención de sobreajuste

El ciclo de entrenamiento se configuró con un máximo de 100 épocas (epochs=100), un tamaño de lote de 4 imágenes (batch=4) y una resolución de entrada de 640x640 píxeles (imgsz=640). Para controlar el descenso del gradiente de manera suave y evitar oscilaciones drásticas en la actualización de los pesos, se definió una tasa de aprendizaje inicial conservadora de 0.001.

El elemento más crítico para la robustez del modelo fue la implementación de un mecanismo de paro anticipado o Early Stopping (patience=20). Al mantener activa la validación cruzada (val=True), el sistema evaluó el rendimiento del modelo al final de cada época utilizando el conjunto de validación aislado. Si la función de pérdida dejaba de disminuir durante 20 épocas consecutivas, el entrenamiento se interrumpía automáticamente. Esto garantizó que el modelo aprendiera a generalizar las características de las señales sin llegar al punto de memorizar el ruido específico de las imágenes de entrenamiento (sobreajuste).

- Aumentación de datos dinámica (On-the-fly Data Augmentation)

Para emular la cinemática de la plataforma robótica y robustecer la percepción ante perturbaciones ambientales, se implementó un pipeline de aumentación estocástica ejecutado dinámicamente en la memoria RAM durante cada iteración. A diferencia de los métodos estáticos que saturan el almacenamiento físico, este esquema aplicó transformaciones matemáticas variables a los tensores de imagen milisegundos antes de inyectarlos a la GPU, asegurando que la red nunca procesara la misma imagen exacta dos veces. Se configuraron las siguientes perturbaciones:

1. Fotométrica: Ajuste aleatorio del canal de valor (brillo) en el espacio de color HSV hasta en un $\pm 40\%$ (`hsv_v=0.4`) para simular las transiciones de iluminación a sombras proyectadas en la pista.
2. Geométrica espacial: Rotaciones afines de $\pm 15^\circ$ (`degrees=15.0`), escalados de alejamiento del 50% (`scale=0.5`) y desplazamientos traslacionales del 10% (`translate=0.1`). Esto simuló matemáticamente la inclinación del chasis por baches, la distancia de frenado y las vibraciones mecánicas del sensor.
3. Fusión contextual: Se utilizó la técnica Mosaic (`mosaic=1.0`) para componer cuatro imágenes distintas en un solo cuadro, obligando al modelo a detectar señales en escalas reducidas y en presencia de múltiples distractores visuales.
4. Restricción semántica: Se forzó la desactivación absoluta de la inversión horizontal (`fliplr=0.0`). Dado que las señales de tránsito poseen una direccionalidad estricta (por ejemplo, una flecha de "vuelta a la derecha"), permitir un espejado habría corrompido semánticamente el conjunto de datos y provocado colisiones inminentes en la lógica de navegación del robot.

Dinámica de ejecución del algoritmo

Durante el tiempo de ejecución, el script operó bajo un flujo de trabajo altamente secuencial y optimizado. Inicialmente, el entorno instanció el modelo cargando los pesos preentrenados (`yolov8n.pt`) directamente en la memoria de la unidad de procesamiento gráfico (`device=0`). Al invocar el método `model.train()`, el sistema interpretó el archivo `data.yaml` para localizar los directorios del dataset y comenzó el proceso iterativo. En cada época, el procesador (CPU) se encargó de cargar lotes de cuatro imágenes (`batch=4`) desde el almacenamiento hacia la memoria RAM, donde aplicó instantáneamente las transformaciones matemáticas definidas (rotación, brillo, escalado y Mosaic). Estos lotes aumentados dinámicamente fueron transferidos de inmediato a la GPU para la etapa de predicción. Tras estimar las cajas delimitadoras, la red calculó su margen de error frente a las etiquetas de validación creadas en Roboflow y ejecutó el algoritmo de backpropagation para ajustar los pesos neuronales. Este ciclo se repitió ininterrumpidamente hasta que el mecanismo de Early Stopping detectó el estancamiento de la función de pérdida en el set de validación, deteniendo el proceso de forma autónoma y exportando el archivo de pesos óptimos (`best.pt`), el cual quedó listo para su posterior integración en el nodo suscriptor de ROS 2.

Resultados

Validación Experimental e Inferencia en Tiempo Real

Para corroborar la eficacia del entrenamiento y la capacidad de generalización de la red neuronal, se ejecutaron pruebas de inferencia en tiempo real utilizando los pesos óptimos exportados por el modelo (best.pt). Esta fase de evaluación física se documentó mediante la grabación de video, en las cuales se sometió al sistema a condiciones operativas dinámicas.

Durante la ejecución, el modelo demostró un rendimiento analítico sobresaliente. El sistema logró identificar, delimitar y clasificar las siete categorías de señales de tránsito con un nivel de confianza (confidence score) consistentemente superior al 80% en las detecciones cuadro por cuadro.

Este alto grado de certidumbre empírica valida de manera definitiva las decisiones metodológicas adoptadas durante el pipeline. El hecho de que el modelo mantenga una precisión superior al 80% durante un flujo de video en vivo, afrontando variaciones reales de distancia, ángulos de visión asimétricos y fluctuaciones lumínicas del laboratorio, confirma que la estrategia de Aumentación de Datos Dinámica en RAM evitó con éxito el sobreajuste. El algoritmo no se limitó a memorizar el dataset base, sino que aprendió a generalizar los patrones morfológicos invariantes de cada señal.

 deteccion señales de transito.mp4

Conclusiones

El desarrollo de este proyecto demostró exitosamente la viabilidad de implementar un sistema de percepción visual avanzado en tiempo real para la plataforma móvil Puzzlebot, superando las restricciones de procesamiento del hardware embebido mediante una arquitectura distribuida en ROS 2. La integración del modelo YOLOv8 Nano, operando en conjunto con el puente de datos cv_bridge hacia una estación de trabajo local, validó la estrategia de descarga computacional (computational offloading), logrando mantener una inferencia fluida y de baja latencia sin saturar el microcontrolador del vehículo.

Los resultados experimentales, evidenciados por un nivel de confianza superior al 80% en la detección en video, confirman que las decisiones tomadas durante el pipeline metodológico fueron correctas. La construcción de un dataset propio capturado directamente con la cámara del robot cerró la brecha de dominio visual, mientras que la implementación de hiperparámetros de regularización, específicamente el paro anticipado (Early Stopping) y la aumentación de datos estocástica y dinámica en RAM, previno el sobreajuste. El modelo demostró no solo memorizar los datos, sino generalizar con éxito los patrones morfológicos de las siete señales de tránsito ante escenarios dinámicos y condiciones lumínicas fluctuantes. Finalmente, este entregable sienta unas bases sólidas y altamente confiables para la siguiente fase del reto. Al contar con un nodo de percepción visual robusto, el sistema está preparado para acoplarse directamente con el control cinemático. Este desarrollo refleja la esencia práctica de la ingeniería en robótica y sistemas digitales, fusionando el procesamiento de software de alto nivel con la respuesta mecánica del hardware. El trabajo futuro se centrará en utilizar las coordenadas espaciales de las cajas delimitadoras detectadas para retroalimentar el controlador proporcional (P) de lazo cerrado, permitiendo que el Puzzlebot transite de la simple percepción a la navegación autónoma puramente reactiva.

Referencia:

- [1] Jocher, G., Chaurasia, A., & Qiu, A. (2023). YOLO by Ultralytics (Version 8.0) [Computer software]. <https://github.com/ultralytics/ultralytics>
- [2] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779-788.
- [3] Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on Image Data Augmentation for Deep Learning. Journal of Big Data, 6(1), 1-48.
- [4] Computer Vision- Dr Cesar Torres Huitzil GoogleColab.Google.com.
https://colab.research.google.com/drive/1e5Z82Gca0yfptq3Y2j8RO6e0oZsUmt_A#scrollTo=j9DZrxWkFgxV